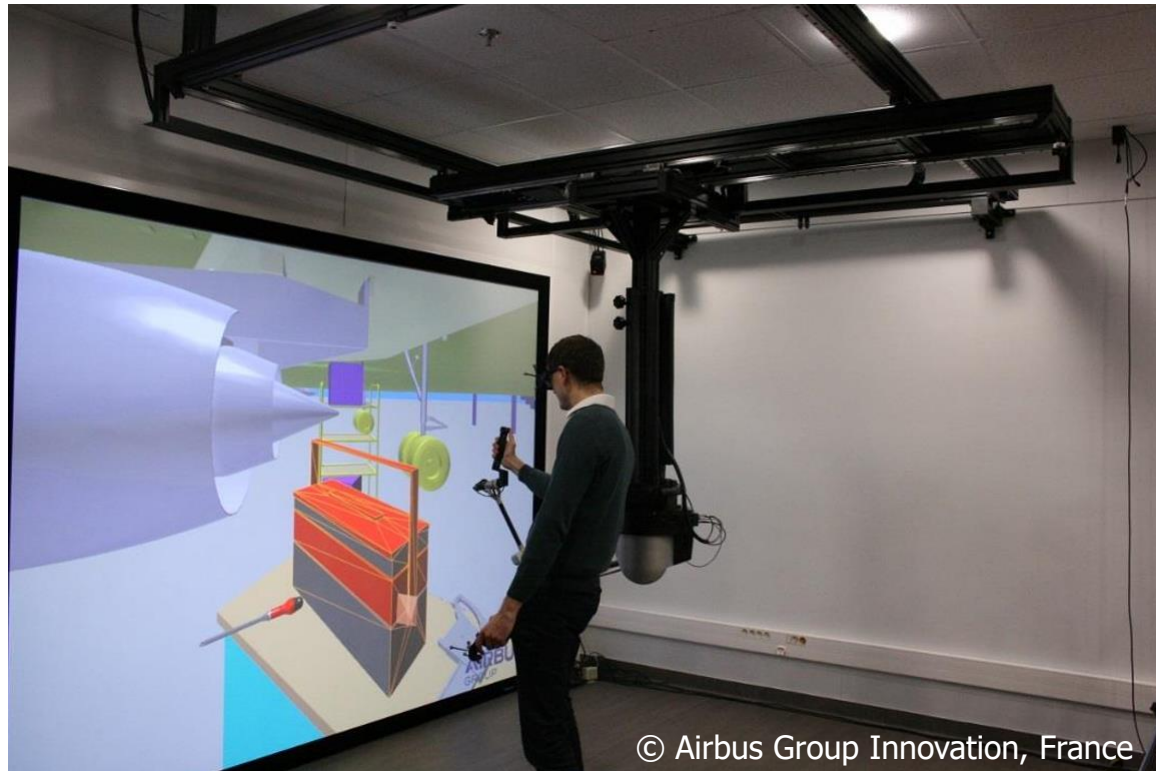


VirtuoseAPI: Common software driver for all Virtuose devices – A tutorial –



- Fundamentals of haptics
 - Definitions
 - Use cases for professional force-feedback
 - Virtual coupling
 - Impedance vs. admittance
- Specifics of HAPTICON products
 - Available control modes
 - Safety procedures
 - Management of coordinate systems
- Using the Virtuose API
 - Programming principles
 - A simple impedance loop
 - A simple admittance loop
- The “standard demonstration”

- Fundamentals of haptics
 - Definitions
 - Use cases for professional force-feedback
 - Virtual coupling
 - Impedance vs. admittance
- Specifics of HAPTICON products
 - Available control modes
 - Safety procedures
 - Management of coordinate systems
- Using the Virtuose API
 - Programming principles
 - A simple impedance loop
 - A simple admittance loop
- The “standard demonstration”

- “Haptics” = stimulating the sense of touch
- Two very different modalities
 - “Tactile feedback” = stimulating the skin
 - Kinaesthetic feedback / “force-feedback” = stimulating the body motor functions (muscles and tendons)



“Operator”



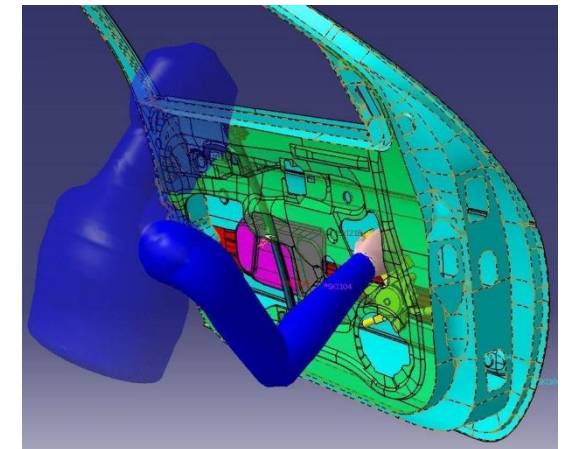
“Haptic device”



“Interface”



“Real process”



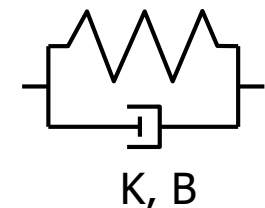
“Virtual process”



Use cases for professional force-feedback

- Industry
 - Assembly validation (virtual process)
 - Ergonomic assessment (virtual process)
 - Remote operation/maintenance (real process)
- Healthcare
 - Surgery training (virtual process)
 - Telemedicine (real process)
 - Co-manipulation / haptic augmentation (real/virtual process)
- Science
 - Psychophysical experiments (real process)
 - Molecular docking (virtual process)
- Culture
 - Virtual worlds (virtual process)
 - Physics experiments (real/virtual process)

- How to interface the device with the process (real or virtual)?
 - *First approach*
 - Set the position of the process to that of the device, and send back the force (computed or measured)
 - In case of a simulated process: only possible with the penalty method (impossible to compute an impact force in that case)
 - Sometimes called “impedance” control
 - Simple but difficult to stabilize
 - *Second approach*
 - Transform the user’s action into a linear force added to the process
 - Send back the updated position transformed into a force
 - Sometimes called the “proxy” method
 - Introduces a position/force transformation “virtual coupling”
- The “virtual coupling”
 - Is a **simulated spring/damper** system (parameters K “stiffness” and B “damping”)
 - Consumes energy, and thus reduces instabilities
 - Is not always easy to compute (especially in rotation)



Impedance vs. admittance (1)

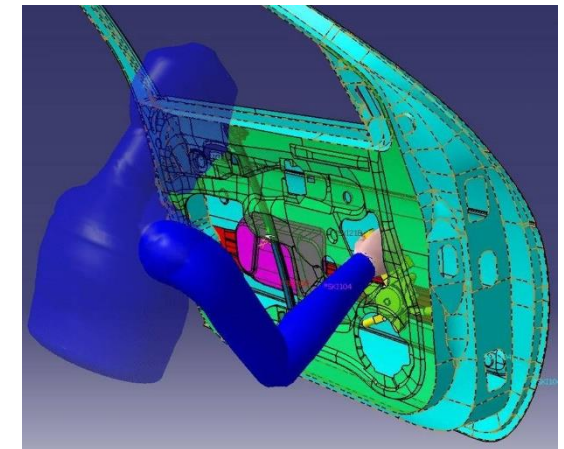
- In **impedance** control
 - The input to the process is a position (the position of the device)
 - The output of the process is a force (the sum of all forces applied to the virtual object, or the measurement of a force sensor, etc.)



Position (& speed)



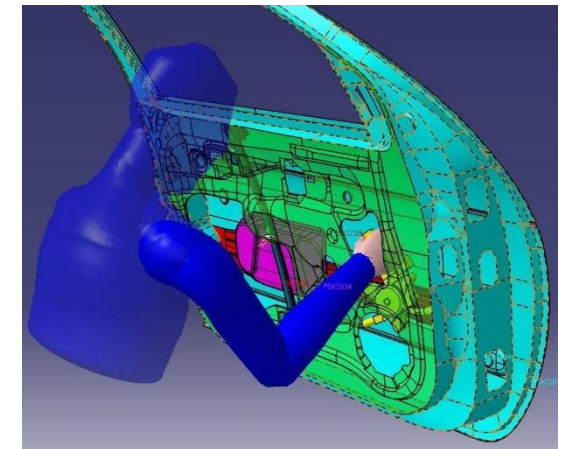
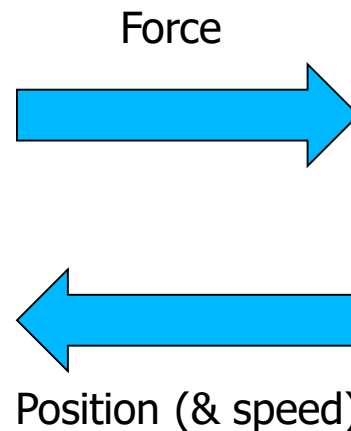
Force



Impedance control is the naive interpretation
of "force-feedback"

Impedance vs. admittance (2)

- In **admittance** control
 - The input to the process is a force (the force applied by the user on the device)
 - The output of the process is a position (the updated position of the virtual object, or the measured position of the TCP)



In admittance control, the haptic device can be seen as a mechanical constraint: $F = f(X)$

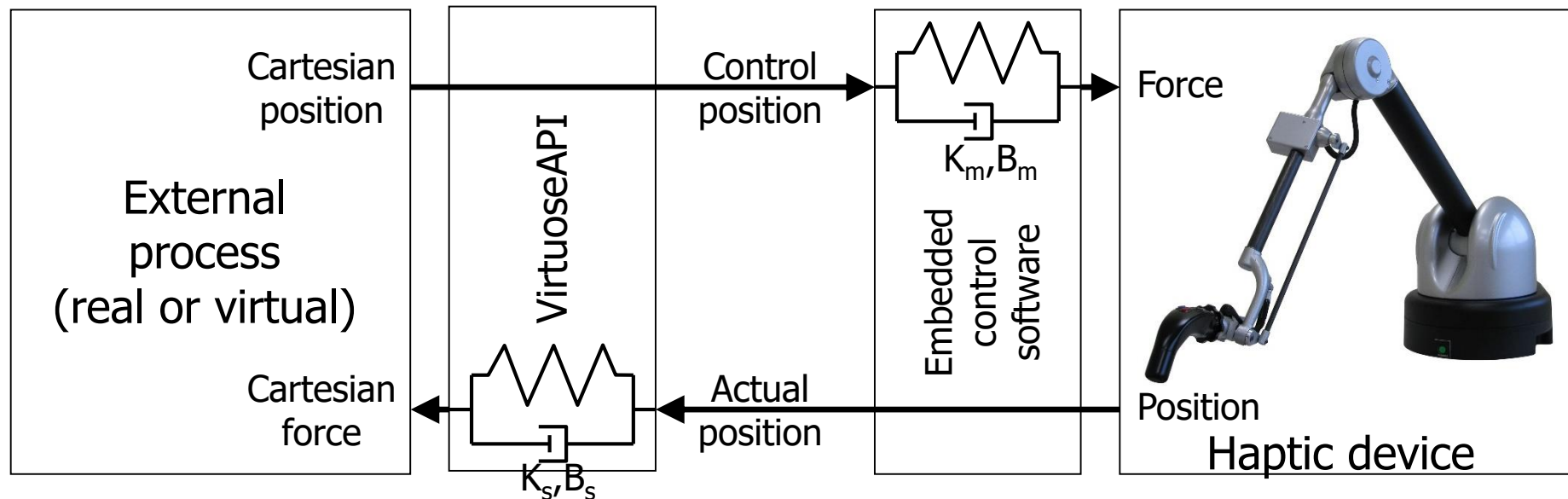
Impedance vs. admittance (3)

- Thanks to the **virtual coupling**, both control modes can be used with all kinds of processes
 - But adding many virtual couplings will jeopardize transparency!
- In theory, admittance control is possible only with a force sensor
 - In practice, a force sensor is not needed if the device/process is **backdrivable**
- There exists an alternative control mode: **position/position**
 - The position of the device is sent to the process
 - The updated position of the process is sent back to the device

- Fundamentals of haptics
 - Definitions
 - Use cases for professional force-feedback
 - Virtual coupling
 - Impedance vs. admittance
- **Specifics of HAPTION products**
 - Available control modes
 - Safety procedures
 - Management of coordinate systems
- Using the Virtuose API
 - Programming principles
 - A simple impedance loop
 - A simple admittance loop
- The “standard demonstration”

Available control modes (1)

- All HAPTION products support impedance control
- They also support **admittance-like** control
- There is no force sensor; the user-applied force is evaluated based on the measurement of the motor currents
- The admittance-like control is actually a **position/position** control:



- In the Virtuose API, it is called "COMMAND_TYPE_VIRTMECH"
- The ratio " K_m/K_s " is the force scaling factor, or simple "force factor"

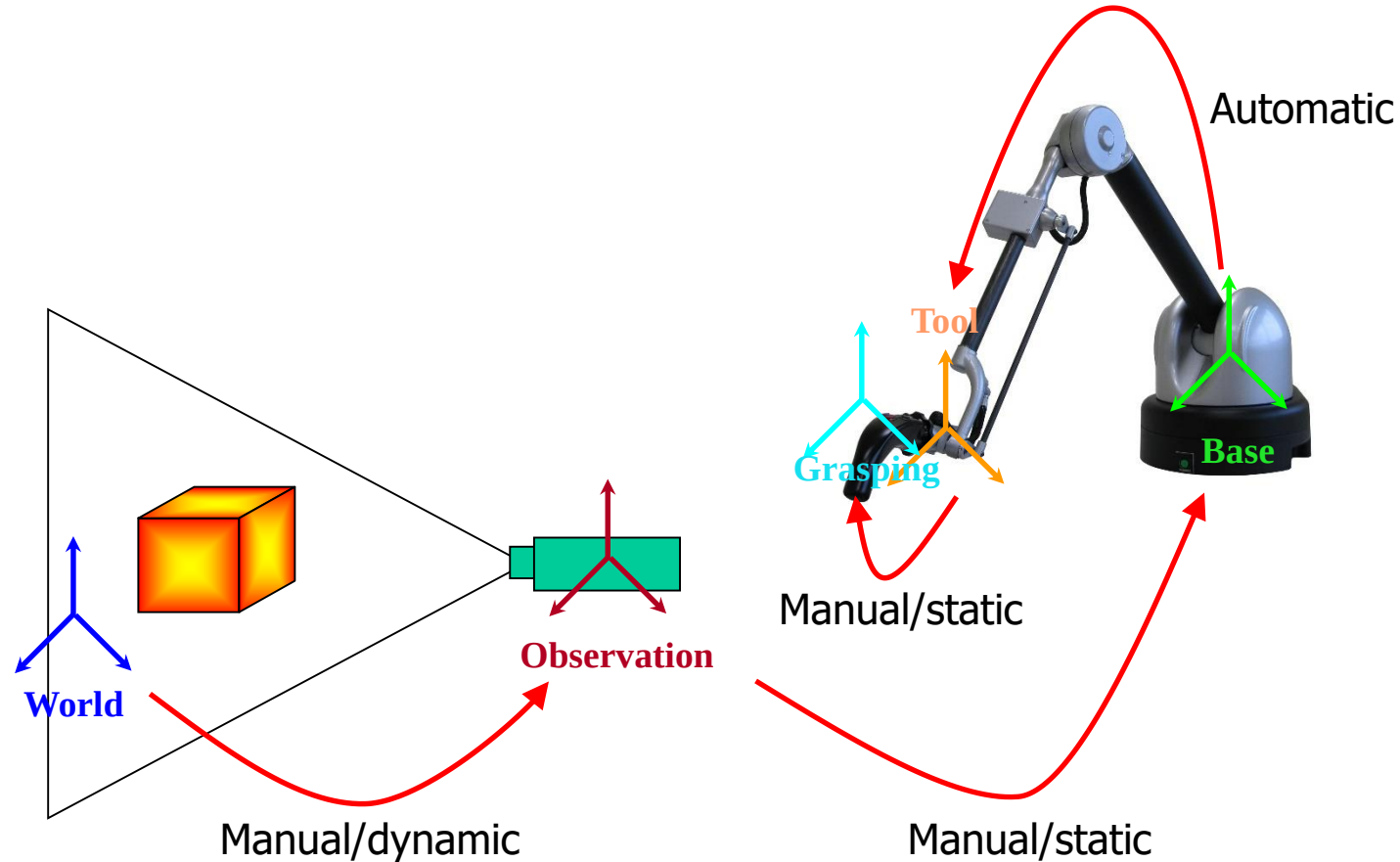
- In control mode "COMMAND_TYPE_VIRTMECH", additional functionalities are available:
 - Freezing of degrees-of-freedom in Cartesian space = "virtual fixtures"
 - Prismatic joint
 - Ball-and-socket
 - Plane constraint
 - Translations or rotations only
 - 6-dof spline
 - Force and position scaling
 - Additional safety level based on energy monitoring
 - Change of control point (of tool, of virtual object)
 - Automatic calculation of spring-damper parameters
 - Simple haptic textures (similar to "bump mapping")
- See the "standard demonstration"...

- Force-feedback can be activated **only if all following conditions** are valid:
 1. The external emergency stop button(s) is(are) connected
 2. The emergency stop button(s) is(are) released
 3. The device has been calibrated in the right position
 4. No hardware failures have been detected
 5. No failure has been detected in the embedded software ("watch-dog")
 6. The front switch is on the "ON" position
 7. All joints are away from the joints limits
 8. The device is connected to a simulation running in real-time ("watch-dog")
 9. The user is holding the handle firmly ("dead-man switch")
 10. The data sent by the simulation is not dangerous ("energy monitoring", only in control mode "COMMAND_TYPE_VIRTMECH")
- In the last case, force-feedback can be turned on again only if the energy goes below the threshold for more than 10 seconds, and if the user has acknowledged the error by:
 - Releasing the handle
 - Grasping the handle again

Management of coordinate systems (1)

- The embedded software knows 5 different coordinate systems:
 - The “base frame” is attached to the base of the device
 - The “tool frame” is attached to the base of the device handle
 - The “world frame” defines the coordinate system of the simulation
 - The “observation frame” defines the position of the camera inside the simulation viewer
 - The “grasping frame” defines the point of grasping of the virtual object with respect to its center of mass
- Using the Virtuose API, you can modify:
 - The position of the “base” with respect to the “observation” (but only once when you start the simulation); it describes the position of the device with respect to the physical display
 - The position of the “observation” with respect to the “world” (dynamically); it describes the movements of the camera, and ensures that the device axes are always coherent with the movements on the screen
 - The “grasping frame” (every time you attach to a new object)

Management of coordinate systems (2)



- Fundamentals of haptics
 - Definitions
 - Use cases for professional force-feedback
 - Virtual coupling
 - Impedance vs. admittance
- Specifics of HAPTICON products
 - Available control modes
 - Safety procedures
 - Management of coordinate systems
- **Using the Virtuose API**
 - Programming principles
 - A simple impedance loop
 - A simple admittance loop
- The “standard demonstration”

- Basic properties of the VirtuoseAPI
 - Generic architecture, supporting all Virtuose devices
 - Multi-platform, multi-device
 - Thread-safe (one thread per haptic device)
 - Simple programming interface
- One header file for C/C++
`#include "virtuoseAPI.h"`
- One static library
virtuoseDLL.lib (Microsoft Windows)
libvirtuose.a (Linux)
- One dynamic library
virtuoseAPI.dll (Microsoft Windows)
virtuoseAPI.so (Linux)
- No loading of the dynamic library until the first call to the API

- A typical program contains **three main parts**

- 1. Initialization*

- Open the connection
 - Choose the control scheme
 - Set control parameters (e.g. scaling factors)
 - Set base frame
 - Enable force-feedback

- 2. Synchronous operation*

- Get device status
 - Apply to external process
 - Send back to haptic device

- 3. Termination*

- Disable force-feedback
 - Close connection
 - Free memory

- Open the connection with a device

```
VirtContext VC = virtOpen(char *address);  
if (VC == NULL)  
{  
    // Error!  
}
```

- Get error status

```
int code = virtGetErrorCode(VirtContext VC);
```

- Get error message

```
char *message = virtGetErrorMessage(int code);
```

- All other functions share the same prototype:

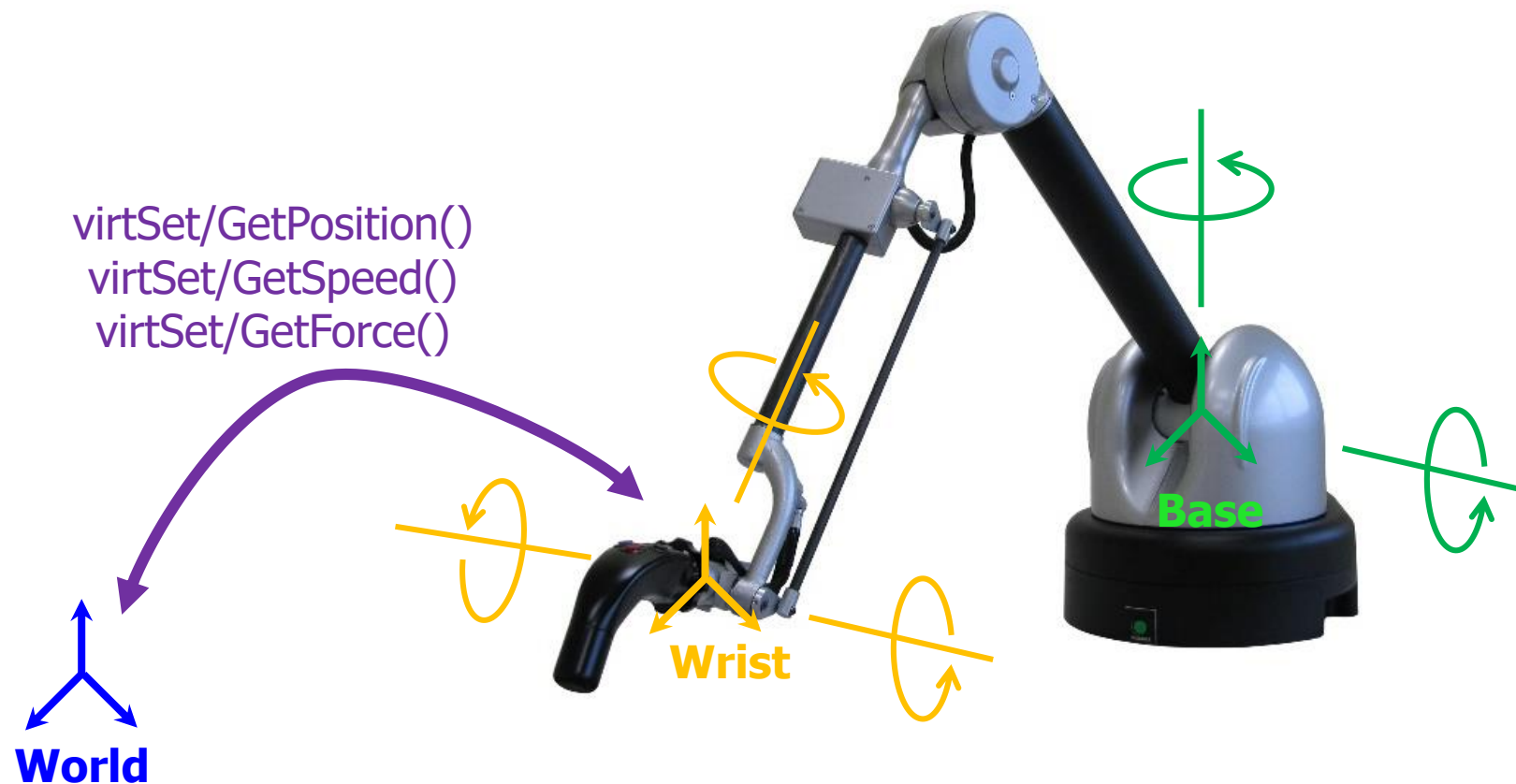
```
int status = virtFunctionName(VirtContext VC, ...);  
if (status)  
{  
    // Error!  
}
```

- All units after international standards
 - Position: meters (m)
 - Angles: radians (rad)
 - Speed: meters per second (m/s)
 - Force: Newtons (N)
 - Torque: Newton-meters (Nm)
 - Mass: kilograms (kg)
 - Inertia: kilograms per square meters (kg/m^2)
 - Translation stiffness: Newtons per meter (N/m)
 - Rotation stiffness: Newton-meters per radian (Nm/rad)
 - Translation damping: Newtons per meter per second (N/(m/s))
 - Rotation damping: Newton-meters per radian per second (Nm/(rad/s))

- All integers of standard "int" type
- All floating-point of standard "float" type
- All vectors and matrices of standard "float []" type
- A position is a float[7]
 - 3 values for the position (X, Y, Z)
 - 4 values for the orientation as a quaternion (Qx, Qy, Qz, Qw)
- A speed is a float[6]
 - 3 values for the velocity (Vx, Vy, Vz)
 - 3 values for the rotation speed (Rx, Ry, Rz)
- A force is a float[6]
 - 3 values for the force itself (Fx, Fy, Fz)
 - 3 values for the torques (Tx, Ty, Tz)

Programming principles (6)

- The control point of the haptic device is the **center of the “wrist”**
- All values are defined relatively to the **world frame**



- Canonical impedance loop program
- External process (real or virtual): tries to follow the position and speed, and measures the interaction force with the environment

```
VirtContext VC;
VC = virtOpen ("192.168.100.53#53210");           // Open connection
virtSetCommandType(VC, COMMAND_TYPE_IMPEDANCE);  // Choose control mode
virtSetPowerOn(VC, 1);                           // Enable force-feedback
while(1)
{
    float position[7], speed[6], force[6];
    virtGetPosition(VC, position);                // Read device position
    virtGetSpeed(VC, speed);                      // Read device speed
    update_process(position, speed, force);        // Call external process
    virtSetForce(VC, force);                      // Write device force
    Sleep(1);                                     // Loop @ 1 kHz
}
virtSetPowerOn(VC, 0);                           // Disable force-feedback
virtClose(VC);                                   // Close connection
```


- Canonical admittance loop program
- External process (real or virtual): applies the force received, and measures its position and speed

```
VirtContext VC;
VC = virtOpen ("192.168.100.53#53210");           // Open connection
virtSetCommandType(VC, COMMAND_TYPE_VIRTMECH);   // Choose control mode
virtSetTimeStep(VC, 0.001f);                     // Specify cycle time
virtAttachVO(VC, mass, inertia);                  // Attach to process
virtSetPowerOn(VC, 1);                            // Enable force-feedback
while(1)
{
    float position[7], speed[6], force[6];
    virtGetForce(VC, force);                       // Read user-applied force
    update_process(force, position, speed);         // Call external process
    virtSetPosition(VC, position);                  // Write device position
    virtSetSpeed(VC, speed);                        // Write device speed
    Sleep(1);                                       // Loop @ 1 kHz
}
virtSetPowerOn(VC, 0);                            // Disable force-feedback
virtClose(VC);                                    // Close connection
```

Using the VirtuoseAPI as scheduler

- After connecting, the Virtuose device is sending one UDP packet every millisecond
- You can use that information as a source of interrupts to schedule your simulation task

```
void virtCallback(VirtContext VC, void *arg)
{
    // Insert your impedance/admittance loop code here!
}

int main()
{
    float timestep = 0.001f;
    VirtContext VC = virtOpen("192.168.100.53#53210");           // Open connection
    virtSetPeriodicFunction(VC, virtCallback, &timestep, NULL);  // Setup callback
    virtStartLoop(VC);                                           // Start update
    getchar();                                                    // Wait for user
    virtStopLoop(VC);                                            // Stop update
    virtClose(VC);                                               // Close connection
}
```

- Fundamentals of haptics
 - Definitions
 - Use cases for professional force-feedback
 - Virtual coupling
 - Impedance vs. admittance
- Specifics of HAPTICON products
 - Available control modes
 - Safety procedures
 - Management of coordinate systems
- Using the Virtuose API
 - Programming principles
 - A simple impedance loop
 - A simple admittance loop
- The “standard demonstration”